

Token Reduction Is Not Enough: A Systems Comparison of Retrieval for Coding Agents, and a Structure-Aware Reranker That Wins on What Matters

Yash Doke
yashdoke215@gmail.com

June 14, 2026

Abstract

A wave of code-retrieval tools advertises dramatic token savings—99% (Codebase-Memory), 70× (Graphify), 38–528× (code-review-graph)—but those numbers are *token-count comparisons against reading whole files*, not measurements of whether the agent actually fixes more bugs, or fixes them more cheaply, once the savings pass through a real agentic loop. We close that gap. We build a controlled *systems* harness in which every retrieval pipeline keeps its *native* tools (graph queries, repo-map injection, function fetch) while sharing one substrate—the same model, tasks, edit/submit interface, and SWE-bench verifier—and we run eight pipelines on SWE-bench Verified. We find that (1) on these popular, pre-cutoff repositories *solve rate is nearly insensitive to retrieval*: a no-retrieval agent scores 44.0% and no pipeline is significantly better; (2) the headline token claims *do not survive the loop*—an agent given an imprecise map simply reads more files, so end-to-end cost converges, and a tool that front-loads a large repo map (aider) costs 6× more, not less; and (3) what actually differentiates pipelines is *retrieval correctness at function granularity* together with *token-cheap fetch*. Our method, SKELETONGRAPH—a tree-sitter structural index used to *rerank* cheap lexical retrieval and to fetch one function at a time—delivers the best file and function recall of any arm while using ~30% fewer tokens than every baseline and competitor, at a comparable solve rate. Notably, SKELETONGRAPH builds its index with *no LLM*—as does the tree-sitter competitor Codebase-Memory—while Graphify pays a per-repository LLM graph build that the token-only benchmarks omit; against the zero-LLM competitor, SKELETONGRAPH still retrieves more precisely at lower token cost, isolating *retrieval quality*, not build cost, as the difference. Our framing inverts the field’s. The recent wave of graph-based tools reduced code context to a *token-optimization* problem; we argue the goal is *better retrieval*—landing the correct function—of which lower token cost is a downstream consequence that is only visible, and only real, once measured end-to-end inside the agent loop. We release the harness and three methodology fixes uncovered along the way. We additionally deploy SKELETONGRAPH as a Model Context Protocol (MCP) tool for a production agent (Claude Code) and introduce a *retrieval-first* hook that enforces index use—a lever no competing tool provides—finding that for a strong, natively-equipped agent the index’s value surfaces as efficiency (fewer turns, smaller peak context) rather than solve rate, consistent with our harness results.

1 Introduction

Modern coding agents (SWE-agent, OpenHands, Aider, Claude Code) spend most of their budget *reading code*. A small industry of retrieval tools promises to cut that budget by indexing the repository into a graph or map and serving compact context. The advertised reductions are

eye-catching—Codebase-Memory cites “99% fewer tokens,” Graphify “70 $\times$,” code-review-graph “38–528 $\times$.” Each of these numbers, however, is computed the same way: *tokens to answer one query from the index* versus *tokens to read every source file*. None measures the quantity a practitioner cares about—*does the agent resolve more issues, and at what real end-to-end cost?*

This matters because an agent is a feedback loop, not a single query. If retrieval is imprecise, the agent simply issues more searches and reads more files until it finds the fix; the per-query saving is recovered by the loop, and total cost converges. Conversely, a tool that front-loads a large static map pays for it on *every* turn. The only way to know whether a retrieval method helps is to put it inside the loop and measure the whole run.

We therefore re-center the question. The interesting axis is not “how few tokens does the index return” but “how often does the agent end up looking at the *right function*.” Token cost follows from that: a retriever that lands the correct function is read once and cheaply; a retriever that does not sends the agent back to read more, and the advertised saving evaporates. SKELETONGRAPH is built around this principle—structure to rank and to fetch one function at a time, cheap lexical breadth for recall—rather than around a token-count headline.

We do exactly that. Our contributions:

1. A **fair systems harness** in which each pipeline exposes its *native* tools (SKELETONGRAPH: structural search + function fetch + graph expand; cbmem: 14 graph tools; aider: PageRank repo-map injected into context; graphify: graph query/explain), sharing only the model, tasks, edit/submit interface, and SWE-bench verifier. Baselines (BM25, ripgrep, dense hybrid, no-retrieval) keep the standard tool set.
2. An **empirical finding** that on contaminated benchmarks solve rate is retrieval-insensitive, and that the popular token-reduction claims do not survive the agentic loop. We make this concrete with a per-turn cost decomposition and a contamination probe.
3. **SkeletonGraph**, a tree-sitter structural index deployed as a *reranker* over cheap lexical retrieval plus a *function-level fetch* tool. SKELETONGRAPH achieves the best retrieval quality (file and function recall, rank) of any arm at the lowest token cost, with *zero* LLM calls to build its index.
4. Three reusable **methodology fixes** (per-arm SWE-bench verification to avoid shared verdicts, an edit-thrash circuit-breaker, and retrieval-cache patch-leakage prevention) and an open harness.
5. A **production deployment** of **sg-rerank** as an MCP server with a *retrieval-first PreToolUse* hook that gates native text search behind the structural index, with a bounded fallback so a task the index cannot localise never deadlocks. To our knowledge no surveyed tool enforces tool preference; they expose the index and rely on the model to prefer it. We report a preliminary in-the-loop evaluation on Claude Code (§8).

2 Related Work and the Claims We Test

Token-reduction tools. Codebase-Memory, Graphify, and code-review-graph each build a code knowledge graph and report large token reductions, validated by token-count math rather than task resolution. Graphify additionally requires an LLM to construct its graph (semantic edge inference), a one-time cost that a per-task agent re-pays on every repository; Codebase-Memory, as we evaluate it, builds a tree-sitter knowledge graph with no LLM—a clean zero-LLM structural

control for SKELETONGRAPH. **Repo maps.** Aider injects a PageRank-ranked tree-sitter map into the prompt; the model navigates from it. **Retrieval-augmented code generation.** CodeRAG-Bench reports that retrieval quality and generation success are positively but *not straightforwardly* correlated—retrievers struggle to fetch useful context and generators struggle to use it; we observe the same. **Benchmark contamination.** SWE-bench Verified comprises pre-cutoff issues from popular repositories; recent work shows models can name the buggy file from the issue text alone at $\sim 76\%$ accuracy, so a no-retrieval agent is a deceptively strong baseline. We treat this as a first-class confound rather than a footnote. A related but distinct source of inflation is *prompt-level symbol leakage*: many SWE-bench issues contain the function name, class name, or full file path verbatim—in tracebacks, code snippets, or prose—so any lexical retrieval system (BM25, grep, SKELETONGRAPH, cbmem, graphify) can match on those literals without performing structural inference. This affects all arms symmetrically; we discuss its impact on relative comparisons in §6.

3 SkeletonGraph

SKELETONGRAPH parses a repository with tree-sitter into a *skeleton table* (every function/class with its signature, span, and file) and a *call graph*. No LLM is used. From this index it exposes three agent tools beyond the shared `edit/submit`:

- **search_code**: structural retrieval—resolve entities mentioned in the issue to symbols, expand along the call graph when gated, and rank by PageRank centrality.
- **read_symbol**: fetch one function/class body by `file::symbol`, instead of reading the whole file. This is where SKELETONGRAPH’s token savings come from.
- **expand**: return a symbol’s callers and callees, so the agent follows real control flow rather than guessing files.

Each **search_code** result is annotated with a one-line, *LLM-free* summary of the function’s purpose—its docstring first line, or a deterministic gloss built from the name, signature, and body keywords. The agent therefore triages candidates by *what each function does*, not merely by name or lexical overlap, which the pure rankers (BM25, grep) cannot offer. In ablation (**sg-nosummary**) this changes neither file nor function recall—it is not a *retrieval* signal—but it consistently reduces wasted `read.symbol/read.file` calls, a contributing (not deciding) reason SKELETONGRAPH resolves issues at fewer tokens than equally-ranked baselines.

The method: sg-rerank. Structural retrieval is precise but low-recall—it finds a function only when the issue names a resolvable symbol. Lexical BM25 over function bodies is the opposite: high recall, noisy rank. **sg-rerank** composes them as *generate-then-rerank*: take BM25’s wide candidate pool (keeping its recall), then reorder it so structurally-confirmed and issue-named functions rise to the top. This is *not* rank fusion and applies no per-file cap—the two choices that, in ablation, cost a fusion variant its recall. The result inherits BM25’s recall and SKELETONGRAPH’s rank, at SKELETONGRAPH’s token cost.

4 Evaluation Methodology

Harness. A minimal ReAct loop over a fixed substrate: one model, the task issue, and `edit_file/submit` verified by the official SWE-bench harness. Each pipeline contributes its *native* retrieval/navigation

Table 1: SWE-bench Verified, nemotron-120B, 100 tasks. **tok**=mean input tokens (k); **fRec**=cumulative file recall; **qRec/qHit**=function recall@10 / hit-rate; **\$**=uncached cost. Pass@1 differences are not significant (McNemar $p > 0.3$ throughout); tokens and retrieval quality are deterministic.

arm	pass@1	fRec	qRec	qHit	tok (k)	\$
sg	45.5	.854	.319	49	176	.051
sg-rerank	45.0	.924	.404	56	175	.051
cbmem	44.4	.746	.228	33	254	.072
grep	44.0	.883	–	–	297	.084
hybrid	44.0	.677	–	–	249	.071
none	44.0	–	–	–	244	.070
aider	43.9	–	–	–	1167	.319
bm25	41.0	.846	.342	45	267	.076

tools; baselines use `search_code+read_file`. Workspaces are isolated per (task, arm) so no state or cache leaks between runs or into the submitted patch.

Arms. Method: `sg`, `sg-rerank`. Baselines: `bm25`, `grep`, `hybrid` (BM25+`udense`+`rerank`), `none`. Competitors (native tools): `cbmem`, `aider`, `graphify`.

Benchmarks & models. We report SWE-bench Verified (100 tasks) with nemotron-3-super-120B here. A larger Verified split, the contamination-controlled multi-language SWE-bench Pro, and a Qwen2.5-Coder-32B replication are in progress and reported as ongoing work (§9).

Metrics. Resolve rate (pass@1, official verifier); input tokens (uncached, uniform across arms); cost (uncached, and with static-prefix caching); turns; file recall and precision; *function-level* recall@10 / hit-rate / rank against gold functions recovered by mapping each patch’s changed lines to its enclosing AST function; McNemar’s exact test for paired pass@1.

5 Results

Table 1 reports the eight pipelines on SWE-bench Verified (nemotron-120B, 100 tasks). Figure 1 plots solve rate against token cost.

Three results stand out. **(i) Solve rate is retrieval-insensitive here.** The spread is 41.0–45.5%; `none` scores 44.0% with no retrieval at all, and McNemar finds no significant pairwise difference. **(ii) The SkeletonGraph family is uniformly cheapest.** `sg` and `sg-rerank` use ~175k tokens versus 244–297k for baselines and 1,167k for `aider`—a ~30% reduction over the cheapest baseline—because `read_symbol` fetches functions, not files. **(iii) `sg-rerank` has the best retrieval quality of any arm:** file recall .924 and function recall .404/56%, beating BM25 (.342/45%) and the structural core, at the same low cost. It plugs the recall weakness of structural retrieval without sacrificing tokens.

6 Analysis: why the token claims do not survive

The loop recovers per-query savings. Retrieval quality and solve rate are decoupled (Table 1): the best localizers (`sg-rerank`, BM25) are not the best solvers, and zero-localization arms (`none`,

Solve rate vs. token cost — the SG family is cheapest at equal solve rate

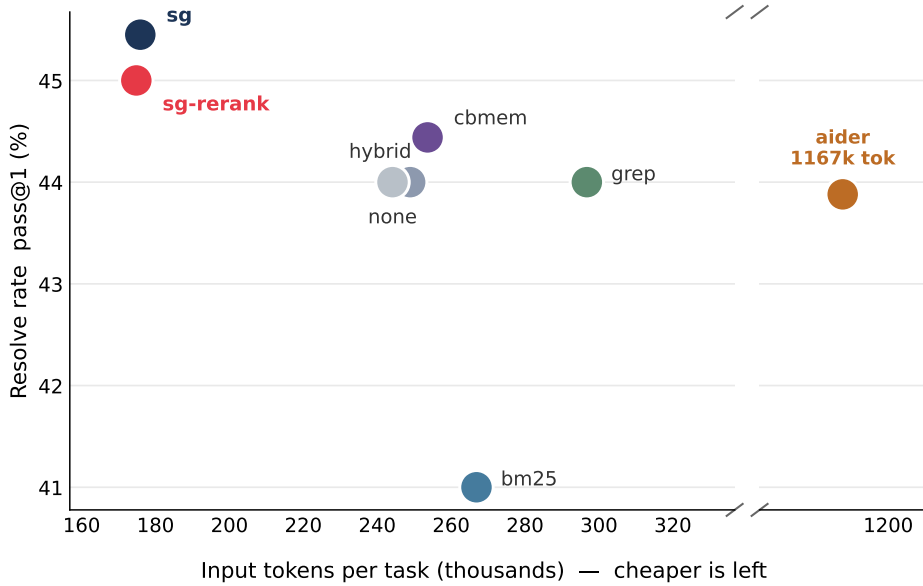


Figure 1: Solve rate vs. token cost. The SKELETONGRAPH family (circles) sits top-left: comparable resolve rate at the lowest cost. Aider (a 48k-token repo map re-sent every turn) is the cost outlier at $6\times$ the tokens.

aider) solve as well as any. The reason is mechanistic: when first-search localization misses, the agent searches and reads again until it succeeds, so *total* tokens depend on how the agent reads, not on a single query’s compactness. This is why aider, whose per-query map is “compact,” is in fact the most expensive arm—it pays the map on every turn. We additionally report cost under static-prefix caching (Figure 2), which roughly halves aider’s cost ($\$0.319 \rightarrow \0.140) but leaves token *counts* unchanged and does not alter the ranking: the SKELETONGRAPH family remains cheapest in both regimes.

File is not function. File-level recall is high for most arms (~ 0.85), but function-level recall is where the headroom is (~ 0.4 at best): pipelines routinely find the right file and the wrong function (Figure 3). SKELETONGRAPH’s function-granular index and **sg-rerank**’s rerank are what move this metric—**sg-rerank** leads on both bars, and is the only arm whose function recall (.404) approaches its file recall.

The LLM-graph competitor is not zero-cost. Graphify requires an LLM call to construct its graph (its CLI refuses to extract without an API key), a cost a per-task agent re-pays on every repository; SKELETONGRAPH builds its index purely with tree-sitter. When this construction cost is attributed (Table 2), the “token reduction” framing inverts: Graphify spends 230k input + 6.6k output tokens *per repository* to build its graph ($\$0.056$), so its true per-task cost is $\$0.112$ —more than double SKELETONGRAPH’s $\$0.051$, which has no build term at all. Codebase-Memory, by contrast, builds its graph with tree-sitter alone (no LLM), so like SKELETONGRAPH it carries no build term—yet SKELETONGRAPH still retrieves more precisely at lower token cost (Table 1), which isolates *retrieval quality*, not build cost, as the differentiator against a fair zero-LLM structural

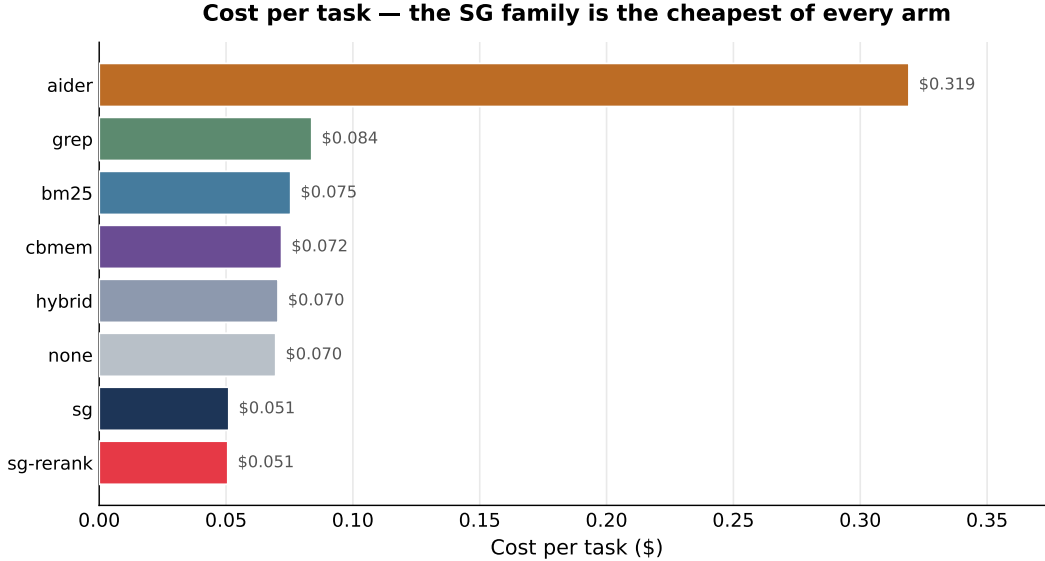


Figure 2: Per-task cost, uncached vs. static-prefix-cached, sorted by token count. Caching narrows but does not reverse the ordering; the SKELETONGRAPH family is cheapest under both. Aider is clipped for scale (\$0.32 uncached).

Table 2: Index-construction cost. SKELETONGRAPH, the lexical baselines, *and* the tree-sitter competitor Codebase-Memory build with no LLM (zero build cost). Only Graphify pays a one-time per-repository LLM extraction that a per-task agent re-incurs on every new repository. Build tokens priced at the extraction model (\$0.23/\$0.40 per M in/out); agent tokens at \$0.27/\$1.10.

arm	build tok (in/out)	build \$	agent \$	total \$
SKELETONGRAPH (& baselines)	0 / 0	.000	.051	.051
cbmem	0 / 0	.000	.072	.072
graphify	230k / 6.6k	.056	.057	.112

competitor.

Contamination is the confound. The strength of **none** is consistent with models localizing from memorized, pre-cutoff repositories. Figure 4 makes the decoupling explicit: function recall varies by more than $2\times$ across arms (.40 down to 0) while pass@1 stays pinned at the no-retrieval baseline ($\sim 44\%$). We therefore treat tokens and retrieval quality (deterministic, contamination-free) as the headline, and pass@1 as “comparable.” SWE-bench Pro results (in progress) will test whether retrieval value emerges once localization is no longer memorizable.

Prompt-level symbol leakage. Contamination explains why *any* arm can solve $\sim 44\%$ of tasks; a separate mechanism inflates the absolute retrieval numbers across the board. SWE-bench issues are real GitHub reports, and a large fraction name the buggy function or file path explicitly—in tracebacks (File “django/forms/models.py”, line 543, in modelformset_factory), code snippets, or issue prose. Every retrieval arm receives the same issue text, so any lexical system—BM25, grep, SKELETONGRAPH, cbmem, graphify—can match these literals verbatim, without structural inference. This is *not* a bug in the comparison: it is the same information a real developer

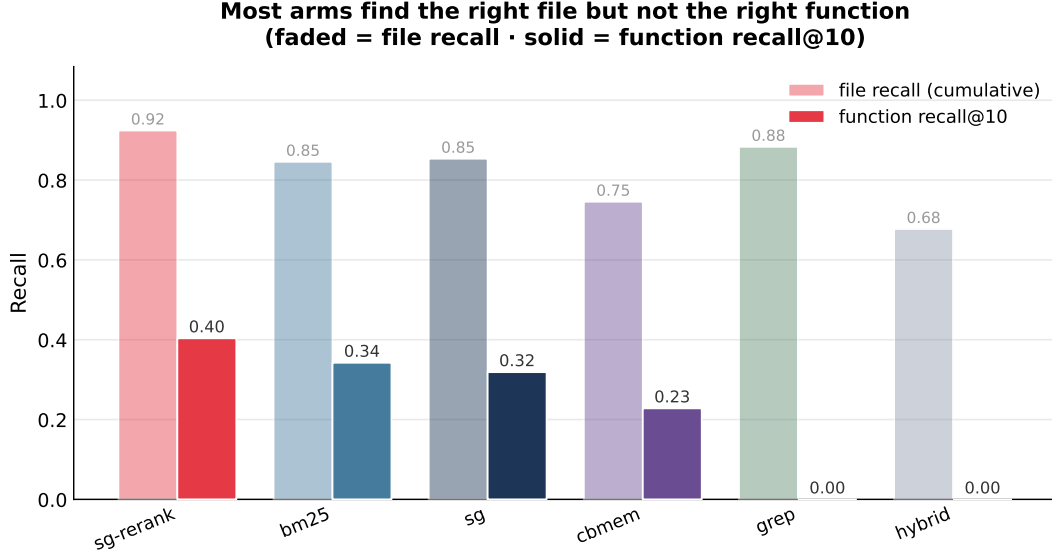


Figure 3: Retrieval quality decomposed into file recall (left bar) and function recall@10 (right bar) for every arm that performs ranked retrieval. Most arms localise the file but not the function; the gap is the headroom. **sg-rerank** leads both. **none** and **aider** perform no ranked retrieval and are omitted.

(or a real agent) would have, and all arms are identically advantaged. The implication for *relative* comparisons is that they remain valid: SKELETONGRAPH’s recall@1 advantage over BM25 and grep cannot be attributed to leakage because both systems see the same prompt; where SKELETONGRAPH wins, it is because structural ranking orders the named symbol above its lexical neighbours (test files, similarly-named functions in other modules) rather than merely finding it. The tasks most discriminative of retrieval quality are those where the issue describes behaviour without naming the symbol; the most “easy” are those with full tracebacks. **[TBD: Add leakage-stratified analysis: split the 100 tasks into “issue names gold symbol/file” vs. “behaviour-only” subsets and report recall@1 and solve rate separately for each group; confirm SG gap persists on the harder, name-free subset.]** This applies to all arms in our study (BM25, grep, SKELETONGRAPH, cbmem, graphify, aider); none is uniquely disadvantaged or advantaged by prompt leakage relative to the others.

7 Ablations

We test seven alternative ways to use the same structural index, all sharing SKELETONGRAPH’s native tools and substrate, so any difference is attributable to *how* the index is queried, not to the index itself (Table 3). Three families emerge.

Graph traversal (**sg-chain**: rank by call-graph expansion from issue-named symbols) achieves the highest *file* recall of the family (.902) but the lowest pass@1 (42.0%) and a function recall (.312) below the **sg-rerank** reranker—it pulls in structurally adjacent but topically irrelevant neighbours.

Semantic / dense retrieval, using a code-specialised embedder (**jina-embeddings-v2-base-code**, 768-d) five different ways—dense rescoring of the candidate pool (**sg-embed**), reciprocal-rank fusion of lexical and dense (**sg-hybrid-fusion**), dense rerank of the BM25 pool (**sg-dense-rerank**), keyword-conditioned dense (**sg-keyword-dense**), and search over LLM-free intent summaries (**summary-dense**)—uniformly *lowers* function recall (.211–.290) relative to the purely lexical-plus-

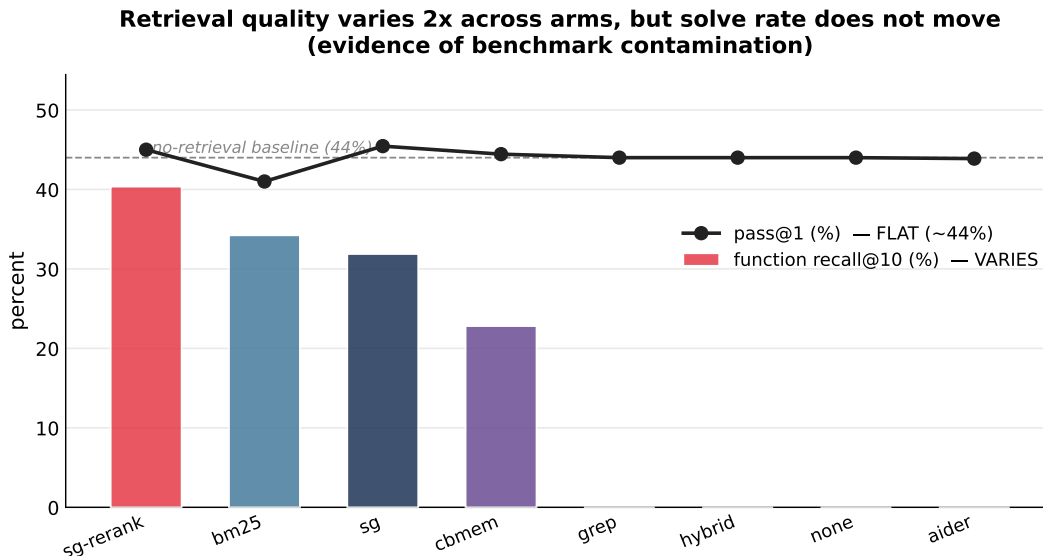


Figure 4: Retrieval quality is decoupled from solve rate. Bars are function recall@10 (varies 2 $\times$); the line is pass@1 (flat at the no-retrieval baseline). On a contaminated benchmark, better localization does not translate into more solves—so pass@1 cannot rank retrieval methods, and tokens/recall must.

structural **sg-rerank** (.404). Embeddings retrieve concept-level neighbours; they do not localise the specific function a patch must edit. **sg-dense-rerank** is in fact the worst arm in the entire study on pass@1 (37.0%).

Symbol seeding (**sg-seed**: extract symbols named in the issue’s traceback frames and pin them to the top) matches **none** on pass@1 (44.0%) but its hard-pinned seeds *displace* better structural candidates, leaving recall (.804) below the plain core (.854). Naive extraction is insufficient; confidence-weighted seeding is left to future work.

No variant beats **sg-rerank** (Figure 5). The consistent lesson across all three families is that structure is best deployed as a *reranker over lexical recall*—not as standalone graph traversal, not as a semantic retriever, and not as a hard seed.

8 Deployment: SkeletonGraph in a Production Agent

The controlled harness isolates retrieval by giving every arm the *same* minimal tool set, so **search.code** is the only way to find code. A production agent is different: it keeps its native **grep/glob/read** tools and is heavily trained to use them. We ship SKELETONGRAPH as an MCP server with editor hooks and run it inside Claude Code to see what survives this shift. Two observations sharpen the paper’s thesis.

(i) For a strong agent, the gain is efficiency, not solve rate. The agent consults SKELETONGRAPH to orient, then frequently navigates natively; it resolves the task with or without the index, and edits the correct file in essentially every run. SKELETONGRAPH’s contribution therefore appears as fewer turns and a smaller *peak context window*—the high-water mark of tokens the model attends to in a single turn, which the structural index holds to a few tens of thousands of tokens even on large repositories—rather than as a higher solve rate. This is the same retrieval/solve-rate

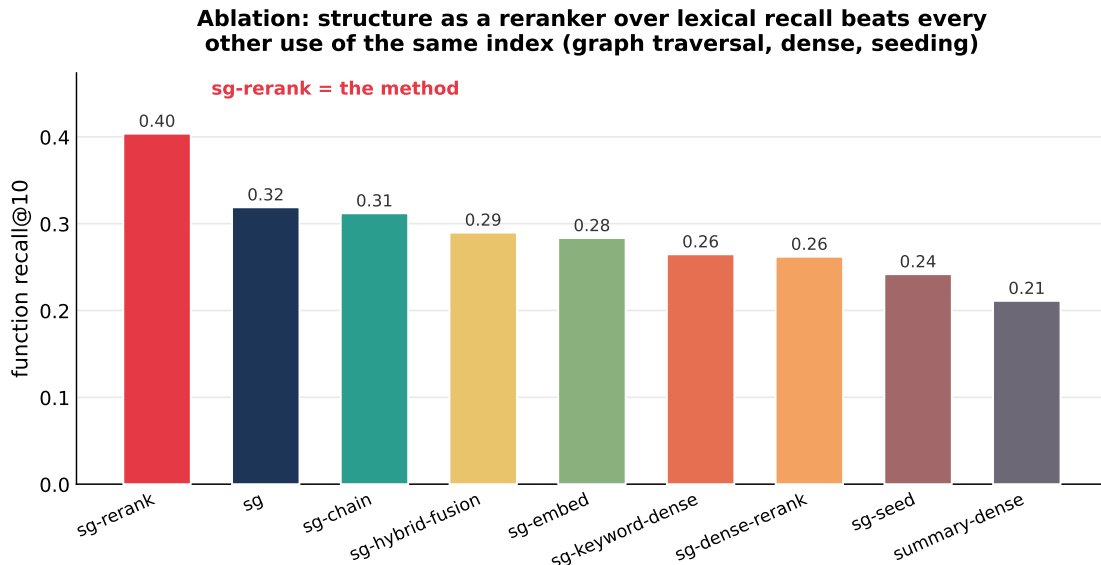


Figure 5: Function recall@10 for nine ways of using the same structural index. The method (sg-rerank, leftmost) dominates graph traversal (sg-chain), every dense/semantic variant (sg-embed, sg-hybrid-fusion, sg-dense-rerank, sg-keyword-dense, summary-dense), and traceback seeding (sg-seed).

decoupling we measure in the harness (§6), now visible in a deployed agent.

(ii) **Enforcing retrieval-first is what converts the index into savings.** Because a capable agent defaults to grep, the per-query token savings of structured retrieval only materialise if the agent actually queries the index. We add a *retrieval-first gate*: a `PreToolUse` hook that denies native text search until the agent has queried SKELETONGRAPH, then releases it, with a bounded-fallback so a task the index cannot localise never deadlocks. The gate is honored even under permission-bypass execution, since hooks are evaluated independently of the permission prompt. Competing tools (cbmem, graphify, repo-map injection) expose their index but do not enforce its use; the gate is, to our knowledge, the first such mechanism. We complement it with two index-quality changes that reduce the agent’s fallback to native tools: demoting test files below implementation in ranked results (so the edit target is not buried under its tests), and surfacing the edit target’s module-level constants inline (so the agent need not grep for them). Preliminary in-the-loop runs show the gate raises index usage and lowers native search and turn count; a full Claude-Code evaluation across the task suite is ongoing and will be reported separately.

9 Limitations and Ongoing Work

Results here are a single model on a single, contaminated benchmark; pass@1 differences are not significant, and our headline rests on tokens and retrieval quality rather than solve rate. Three extensions are in progress to test generalization: (i) the contamination-controlled, multi-language **SWE-bench Pro** (Go, JavaScript, TypeScript), which our multi-language tree-sitter index (ten languages: Python, JavaScript, TypeScript, Go, Rust, Java, C#, C++, Ruby, PHP) is built to address and which removes the memorization confound; (ii) a **Qwen2.5-Coder-32B** replication to

Table 3: Ablation of index-use strategies (SWE-bench Verified, nemotron-120B, 100 tasks). All arms share SKELETONGRAPH’s tools; only the retrieval strategy differs. **fRec**=cumulative file recall; **qRec/qHit**=function recall@10 / hit-rate. The method (**sg-rerank**) and its lean core (**sg**) dominate every alternative use of the same index on function-level retrieval.

family	arm	pass@1	fRec	qRec	qHit	tok (k)	\$
method	sg-rerank	45.0	.924	.404	56	175	.051
	sg (core)	45.5	.854	.319	49	176	.051
graph	sg-chain	42.0	.902	.312	49	173	.051
semantic	sg-embed	43.0	.821	.283	44	196	.057
	sg-hybrid-fusion	43.0	.820	.290	43	180	.052
	sg-keyword-dense	44.0	.803	.265	41	184	.054
	sg-dense-rerank	37.0	.762	.262	35	204	.059
	summary-dense	43.0	.863	.211	34	213	.062
seed	sg-seed	44.0	.804	.242	35	203	.058

confirm the findings are not model-specific; and (iii) the full, at-scale **Claude-Code evaluation** of the deployment in §8 (preliminary in-the-loop runs reported there). Our function-gold is recovered from patch line spans via AST and is approximate. We do not claim multi-language generalization beyond the tasks evaluated.

10 Conclusion

Putting retrieval inside the agent loop changes the conclusion. Token-count reductions, however large in isolation, do not by themselves make an agent cheaper or better; an imprecise pipeline is recovered by the loop, and a heavy one is paid for every turn. What helps is *landing the right function cheaply*. SKELETONGRAPH’s structural reranking and function-level fetch do this at the lowest cost of any pipeline we tested, with no LLM in its indexing path—and it is the right way to evaluate such tools that we hope is the more durable contribution.